



Namal University, Mianwali

Department of Computer Science

Course Name:
CSC-271 - Database Systems

Project Proposal

AI-Based Task Manager & Smart Scheduler

Prepared By

Mishaal Aqdas	(NUM-BSCS-2024-35)
Muhammad Imran	(NUM-BSCS-2024-50)
Khadija Tul Kubra	(NUM-BSCS-2024-30)

Submitted To

Mam Asiya Batool

Date: 18 January 2026

Contents

1	Introduction	2
2	Problem Statement	2
3	Objectives	2
4	Scope	3
4.1	Included Features	3
4.2	Excluded from Scope	3
5	Data Collection	4
5.1	Primary Data Sources	4
5.2	Sample and Test Data	4
6	Data Volume Estimates	4
7	Technology Stack	4
8	System Functionality	5
9	User Classes	6
9.1	End User (Standard User)	6
9.2	Administrator	6
10	Conclusion	6
11	References	7

1 Introduction

The AI-Based Task Manager & Smart Scheduler is a mobile application designed to help individuals—particularly students and working professionals—efficiently organize and manage their daily tasks, appointments, and schedules. In an age of increasing personal and professional commitments, the ability to track tasks digitally has become essential for productivity and mental well-being.

From a database perspective, this system relies heavily on structured, persistent, and relational data storage. Every task, user account, reminder, schedule entry, and system log must be stored, retrieved, and updated efficiently. A robust database backend ensures data integrity, fast access, and the ability to support multiple users simultaneously.

This proposal outlines the database design and development plan for the AI-Based Task Manager & Smart Scheduler, covering the data entities, relationships, technology choices, and system functionality required to build a reliable and scalable database system.

2 Problem Statement

Many individuals rely on informal methods—such as paper notes, personal memory, or unstructured digital files—to track their tasks and appointments. These methods are unreliable, difficult to search and retrieve, and do not support features like automated reminders, task rescheduling, or progress analytics.

From a data management standpoint, the core problems are:

- No centralized, queryable data store for personal tasks and schedules.
- No structured relationship between tasks, users, deadlines, and reminders.
- Inability to efficiently retrieve, filter, or sort tasks based on priority, status, or date.
- Lack of persistent storage that survives application restarts or device changes.
- No audit trail or history of task completions and modifications.

The AI-Based Task Manager & Smart Scheduler addresses these issues by implementing a well-structured relational database that centralizes all task-related data and enables fast, reliable data operations.

3 Objectives

The primary objectives of this database project are:

1. Design a normalized relational database schema to store users, tasks, categories, reminders, and schedule entries.
2. Implement efficient queries for task retrieval by date, priority, status, and user.
3. Ensure data integrity through primary keys, foreign keys, and appropriate constraints.

4. Support CRUD (Create, Read, Update, Delete) operations for all major data entities.
5. Enable tracking of task history and completion statistics for progress reporting.
6. Implement role-based access control at the database level to differentiate between regular users and administrators.
7. Optimize database performance using indexing strategies on frequently queried fields.
8. Provide a scalable schema that can accommodate a growing user base without redesign.

4 Scope

The database system will support the following features and data management capabilities:

4.1 Included Features

- User account management: registration, authentication data, and profile storage.
- Task management: creation, editing, deletion, and status updates (pending, in-progress, completed, postponed).
- Category and tag management: organizing tasks under user-defined categories.
- Reminder storage: linking time-based reminders to specific tasks.
- Schedule views: storing and retrieving tasks filtered by daily, weekly, or monthly date ranges.
- Smart suggestions log: storing AI-generated suggestions and whether users acted on them.
- Progress and analytics data: aggregated task completion statistics per user.
- Admin data access: full read/write access to all tables for administrative management.

4.2 Excluded from Scope

- Third-party calendar integration (e.g., Google Calendar sync) is not in scope for this phase.
- Real-time collaborative task sharing between multiple users.
- Cloud-based distributed database deployment (local/single-server deployment only).

5 Data Collection

The data required for this system will be gathered and populated through the following methods:

5.1 Primary Data Sources

- **User Input:** All core data (tasks, categories, reminders, user profiles) is entered directly by users through the application interface and stored in the database via parameterized queries.
- **System-Generated Data:** Timestamps (creation date, modification date, completion date), task status transitions, and reminder trigger logs are auto-generated by the application backend.
- **AI Module Output:** Smart suggestions generated by the AI module will be logged in a dedicated suggestions table, recording the suggestion text, timestamp, context, and user response.

5.2 Sample and Test Data

- Realistic sample data will be manually created to populate the database during development and testing phases, simulating real-world usage patterns across multiple user profiles.
- Data validation rules (e.g., NOT NULL constraints, CHECK constraints on status fields, date format enforcement) will be applied at the database level to ensure only clean data enters the system.

6 Data Volume Estimates

Entity	Estimated Records (per user)	Growth Rate
Users	N/A (global)	~50–250 users in testing
Tasks	50–200	Daily additions
Reminders	30–100	Per task created
Categories	3–10	Occasional
Suggestions Log	10–50	Weekly
Progress Records	1 per user per week	Weekly aggregation

Table 1: Estimated Data Volume per User

7 Technology Stack

The following technologies will be used to design, develop, and deploy the database system:

MySQL was chosen for its strong support for relational data, ACID compliance, wide community support, and ease of integration with Python and mobile backends. MySQL Workbench provides a visual environment that aids in schema design and documentation.

Component	Technology	Purpose
Primary DBMS	MySQL 8.0	Relational database management and SQL query execution
Database Design	MySQL Workbench	ER diagram creation, schema visualization, and forward engineering
Backend Language	Python (Flask)	Application layer connecting the mobile app to the database
ORM / Query Tool	SQLAlchemy	Object-relational mapping and safe parameterized queries
Mobile Frontend	Flutter (Dart)	Cross-platform mobile UI that interacts with the backend API
Version Control	Git / GitHub	Source code and schema versioning for team collaboration
Testing	MySQL Workbench + Postman	SQL query testing and API endpoint validation

Table 2: Technology Stack for System Development

8 System Functionality

The database system will support the following key functional requirements:

1. **User Registration and Authentication:** Store user credentials securely (hashed passwords), enable login validation, and manage user sessions via tokens stored in the database.
2. **Task Creation and Storage:** Insert new task records into the Tasks table, linking each task to a user and category, with fields for title, description, priority, due date, and status.
3. **Task Retrieval and Filtering:** Query tasks by user ID, status (pending/completed/postponed), date range, and priority level using optimized SELECT statements with WHERE clauses and indexes.
4. **Task Update and Deletion:** Support UPDATE operations for editing task details or changing status, and DELETE operations (soft-delete preferred to preserve history).
5. **Reminder Management:** Insert, retrieve, and delete reminder records linked to specific tasks, enabling time-based notification triggers.
6. **Category Management:** Allow users to create and assign custom categories to tasks; support filtering tasks by category.
7. **Task Rescheduling / Postponement:** Update the `due_date` field and log the change in a task history table with timestamp and reason.
8. **Progress and Analytics Queries:** Aggregate task completion rates per user per week/month using GROUP BY and COUNT queries, and store results in a progress summary table.

9. **Smart Suggestion Logging:** Store AI-generated suggestions with context, timestamp, and user action (accepted/dismissed) for future model improvement.
10. **Admin Data Management:** Provide admin-level access to query all users' data, generate system-wide reports, and manage data integrity.

9 User Classes

The system defines two primary user classes with distinct roles and database access levels.

9.1 End User (Standard User)

End users are the primary consumers of the application — students, professionals, and anyone managing personal tasks.

- Database Access Level: Restricted to their own data only.
- Can perform CRUD operations on their own tasks, categories, and reminders.
- Can read their own progress and suggestion history.
- Cannot access other users' records or system-level tables.
- Authenticated via user ID and session token on every database request.

9.2 Administrator

Administrators are responsible for maintaining system health and managing data quality.

- Database Access Level: Full read and write access to all tables.
- Can view and manage all user accounts, including deactivating or deleting accounts.
- Can run system-wide reports and analytics queries.
- Can manage database backups and monitor query performance.
- Authenticated via a separate admin credentials table with elevated privileges.

10 Conclusion

The AI-Based Task Manager & Smart Scheduler database project aims to build a robust, normalized, and efficient relational database system that serves as the data backbone for a modern productivity application. By centralizing task, user, reminder, and schedule data within a well-structured MySQL database, the system will enable fast queries, reliable data integrity, and a scalable foundation for future feature expansion.

The expected impact of this system includes improved data organization and retrieval speed for personal task management, a reusable database design pattern applicable to similar productivity applications, practical experience with real-world database design, SQL querying, and backend integration for all team members, and a foundation that can

be extended to support AI-driven analytics, machine learning data logging, and multi-user collaboration in future iterations.

This project will demonstrate the application of core database system concepts—including normalization, relational modeling, indexing, and access control—in a meaningful, real-world context.

11 References

The following sources were consulted in the preparation of this proposal:

1. Ramakrishnan, R., & Gehrke, J. (2003). *Database Management Systems* (3rd ed.). McGraw-Hill.
2. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database System Concepts* (7th ed.). McGraw-Hill.
3. MySQL Documentation. (2024). *MySQL 8.0 Reference Manual*. Oracle Corporation.
<https://dev.mysql.com/doc/refman/8.0/en/>